System Requirements Specification: Thelawala Bridge Server

1. System Overview

The Thelawala Bridge Server is a real-time middleware backend designed to connect street vendors ("Thelawalas") with local buyers ("Clients"). The system handles user authentication, persistent inventory management, bidirectional real-time location tracking, and an instant request/response order protocol.

2. Actor Profiles & Authentication Flows

The system must support two distinct user roles, each with its own authentication logic.

2.1 Thelawala (Vendor) Authentication

- **Credentials:** thelaId (String, Unique) and password (String, Hashed).
- **Flow:** * Vendor submits credentials to the login endpoint.
    - If the thelaId exists and the password matches, return a JWT (JSON Web Token) to grant access.
    - If the thelaId does not exist, the server must intercept the 404/Unauthorized error and prompt the frontend to ask: *"Account not found. Would you like to register with these credentials?"*
    - If the user agrees, route to a registration endpoint to create the account and grant access.

2.2 Client (Buyer) Authentication

- **Credentials:** phoneNumber (String, Unique) and password (String, Hashed).
- **Flow:** Identical to the Vendor flow. Login attempts with unregistered phone numbers must prompt the user to create a new Buyer account using the provided credentials.

3. Core Vendor (Thelawala) Features

Once authenticated, the server must support the following capabilities for vendors:

- **Inventory Publishing:** The vendor can upload/update a list of items they are carrying for the day (e.g., [{ id: 1, name: "Tomatoes", qty: 20 }]).
- **Location Broadcasting:** The vendor can toggle a continuous broadcast of their live GPS coordinates to the server.
- **Inbound Request Handling:** The server must route incoming buyer requests (containing the buyer's location and requested item) directly to the specific vendor's active socket connection.
- **Order Disposition:** The vendor can accept or reject the buyer's request.
- **Dynamic Routing:** If the vendor accepts, the server must transmit the buyer's exact GPS coordinates to the vendor so they can be pinned on the vendor's map UI.

4. Core Client (Buyer) Features

Once authenticated, the server must support the following capabilities for buyers:

- **Proximity Mapping:** When the buyer grants location access, the server must return a list of all active, broadcasting Thelawalas within a specific radius, including their current inventory.
- **Search by Thela ID:** The buyer can query a specific vendor by their ID.
    - If the vendor is actively broadcasting, return their exact location and inventory.
    - If the vendor's location toggle is off, return a status indicating: *"Vendor is currently offline/not broadcasting."*
- **Search by Item:** The buyer can query a specific string (e.g., "Onions"). The server must filter active vendors and return a list of Thelawalas carrying that item today, along with their locations.
- **Item Requesting:** The buyer can trigger an event to request an item from a specific vendor. The server must pass this request and the buyer's location to the vendor.
- **Request Status Subscriptions:** The buyer must receive a real-time push notification/update when the vendor hits "Accept" or "Reject".

5. Technical Architecture & Protocols

To fulfill the real-time requirements, the server must implement a hybrid protocol approach.

5.1 REST API (HTTP) Used for one-time, state-changing actions:

- POST /api/auth/vendor/login & POST /api/auth/vendor/register
- POST /api/auth/client/login & POST /api/auth/client/register
- POST /api/vendor/inventory (Update daily stock)
- GET /api/search/vendor/:id
- GET /api/search/item/:itemName

5.2 WebSocket (TCP/Socket.io) Used for low-latency, bidirectional streaming:

- Event: vendor_location_update: Vendor streams { lat, lng } to the server.
- Event: get_nearby_vendors: Client requests vendors within a radius. Server responds with live tracking data.
- Event: send_item_request: Client sends { vendorId, itemId, clientLocation: { lat, lng } }. Server routes this strictly to the specified vendorId's socket.
- Event: request_decision: Vendor sends { clientId, decision: 'accepted' | 'rejected' }. Server routes this back to the clientId's socket.

6. Database Schema (High-Level Entities)

- **Vendor Table:** id, thelaId (unique), password_hash, isBroadcasting (boolean), lastLocation (GeoJSON), currentInventory (JSON/Array).
- **Client Table:** id, phoneNumber (unique), password_hash.
- **Active Sessions (In-Memory/Redis):** Mapping of userId to socketId to instantly route direct requests and approvals.